

Yes, exactly. To run a model as large as **Qwen 3.5 9B Q4_0**, it is split up into pieces because of a strict hardware limitation built right into Qualcomm's mobile architecture.

Per the official [llama.cpp Snapdragon Hexagon Developer Documentation](#), Qualcomm's mobile hardware enforces a hard memory ceiling on a single NPU context: [1]

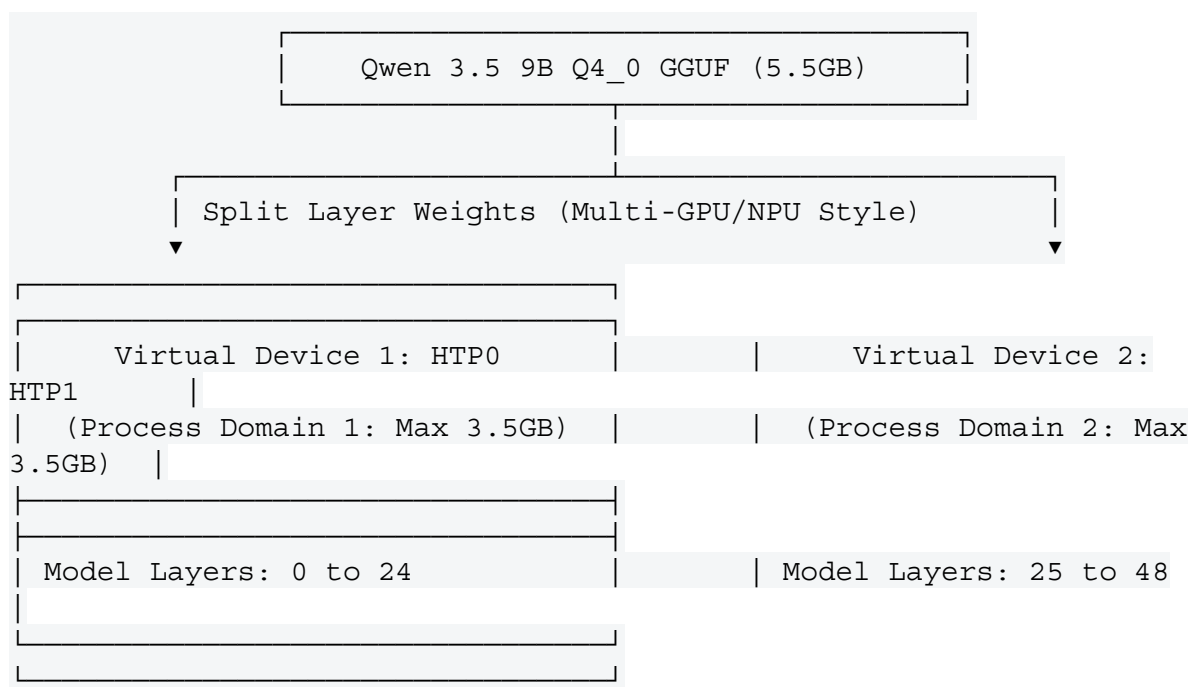
*"Hexagon NPU session (aka Process Domain (PD) in the Hexagon docs) is limited to a memory mapping of **around 3.5GB**."* [1]

Because a 9B Q4_0 model requires roughly **5.5GB to 6GB of VRAM** to fit its weights and KV cache, a single NPU session will instantly throw an out-of-memory error and crash. [1]

To bypass this limit, the system treats your single large model like **multiple virtual backends** and splits it across separate execution slots. [1]

How the Model Splitting Works Natively

To run a large model, the orchestrator splits the layer architecture across independent virtual NPU pipelines (HTP0, HTP1, etc.). [1, 2]



1. Multi-Device Virtualization (HTP0-4)

In Qualcomm's driver setup, the Hexagon session hooks map into multiple virtual hardware IDs.

According to the llama.cpp Snapdragon manual, developers are forced to use multi-GPU layer-splitting strategies to target the NPU: [1, 2]

"In order to map models larger than 3.5GB we need to allocate multiple devices and split the model. For this we're taking advantage of the llama.cpp/GGML multi-GPU layer-splitting support. Each Hexagon device behaves like a GPU from the offload and model splitting perspective." [1]

2. Cross-Process Domain Communication

Behind the scenes, the Android NDK treats HTP0 and HTP1 as two distinct **Process Domains (PD)**.

- When you run inference, the first half of Qwen's mathematical layers execute inside HTP0's memory sandbox.
- The mid-point tensor values are then shipped across to HTP1 via a fast, hardware-level RPC link (Remote Procedure Call) inside the chip, allowing the second half of the layers to finish calculating the token output.

Executing the Split model via Command Line

When you instruct your background daemons to target the model, your internal execution script must explicitly structure the multi-device offload parameters. [2, 3]

For example, using the standard Snapdragon CLI wrappers, a split model run forces the multi-device allocation string (D=HTP0,HTP1): [2, 3]

```
# Direct hardware execution mapping a 9B model over two virtual HTP domains
M=qwen3.5-9b-q4_0.gguf D=HTP0,HTP1 ./run-completion.sh -p
"Translate this text:"
```

What This Means for Your Orchestrator Daemon

Because the model acts like two distinct engines under the hood, your **C++ Daemon** cannot simply call a basic, single-threaded initializer loop.

Your C++ engine must initialize the Qualcomm driver using an array of context requests, registering separate shared memory (ion/memfd) allocations for each half of the model layer blocks to satisfy the 3.5GB hardware boundary constraints. [1]

If you'd like, let me know:

- What **Snapdragon chip variants** you are targeting (e.g., Snapdragon 8 Gen 3 or 8 Elite).

The newer chips support a higher number of parallel hardware channels.

- If you want the exact **C++ initialization block** to create multiple concurrent QNN context handles (Qnn_ContextHandle_t) for a multi-device split. [1, 2]

I can map out the precise implementation logic!

[1] <https://github.com>

[2] <https://github.com>

[3] <https://github.com>